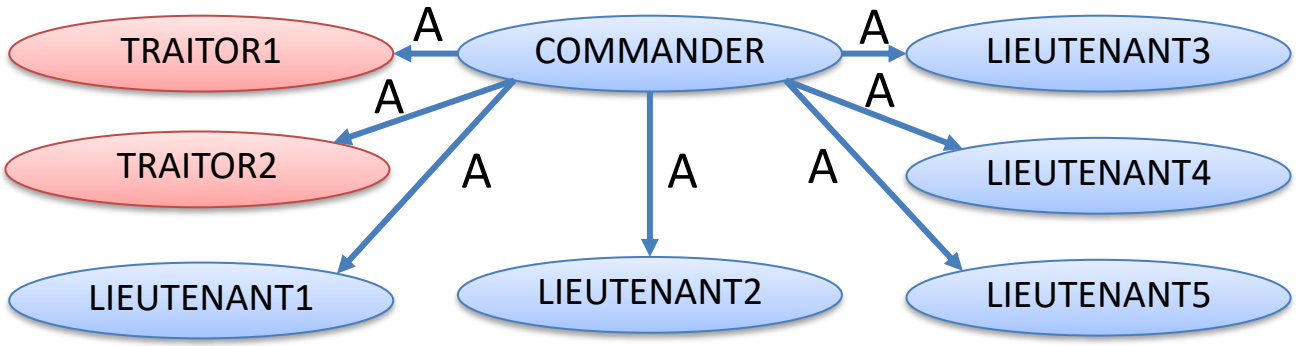
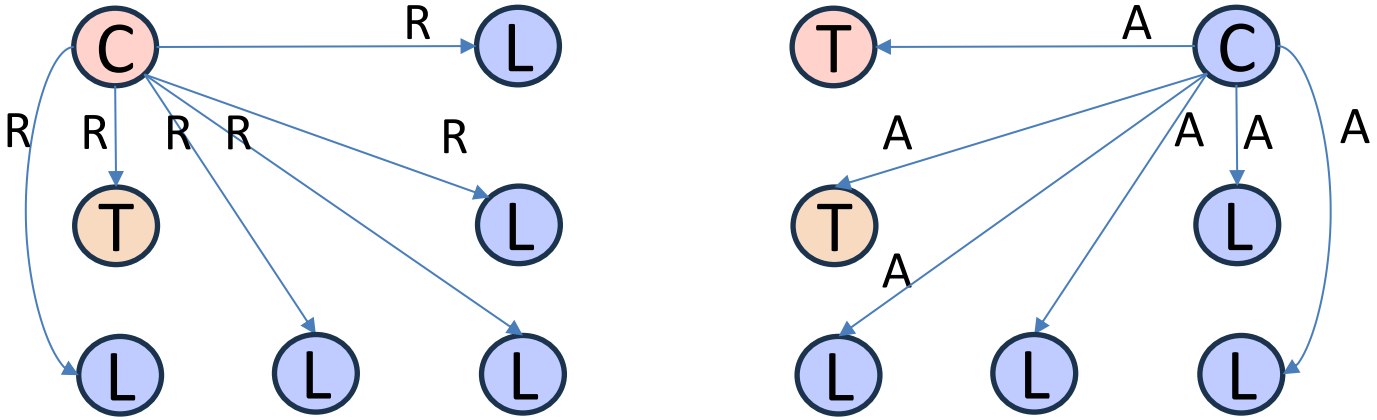
Home Exercise: BFT (1/2)

OM(2): Commander sends A (Attack) to all 7 lieutenants (incl. traitors)

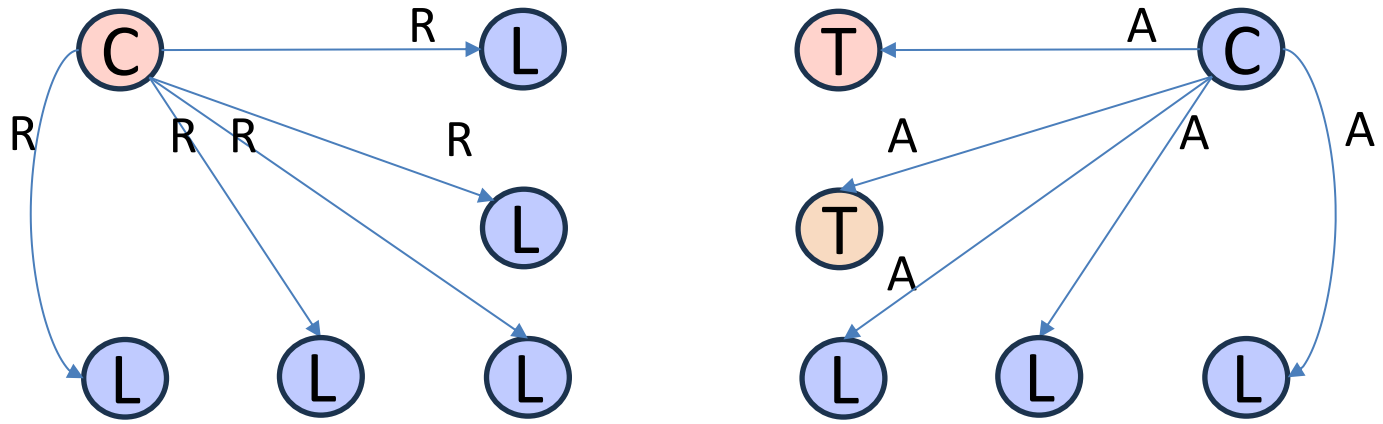


OM(1): Every lieutenant (incl. traitor) acts as commander



Home Exercise: BFT (2/2)

OM(0): Every lieutenant (incl. traitor) relays the message



Lieutenant	Value frm C	Frm L1	Frm L2	Frm L3	Frm L4	Frm L5	Frm T1	Frm T2	Majority Vote
L1	A	-	A	A	A	A	R	R	A
L2	A	A	-	A	A	A	R	R	A
L3	A	A	A	-	A	A	R	R	A
L4	A	A	A	A	-	A	R	R	A
L5	A	A	A	A	A	-	R	R	A
T1	X	X	X	X	X	X	-	X	X
T2	X	X	X	X	X	X	X	-	X

Rollback Recovery

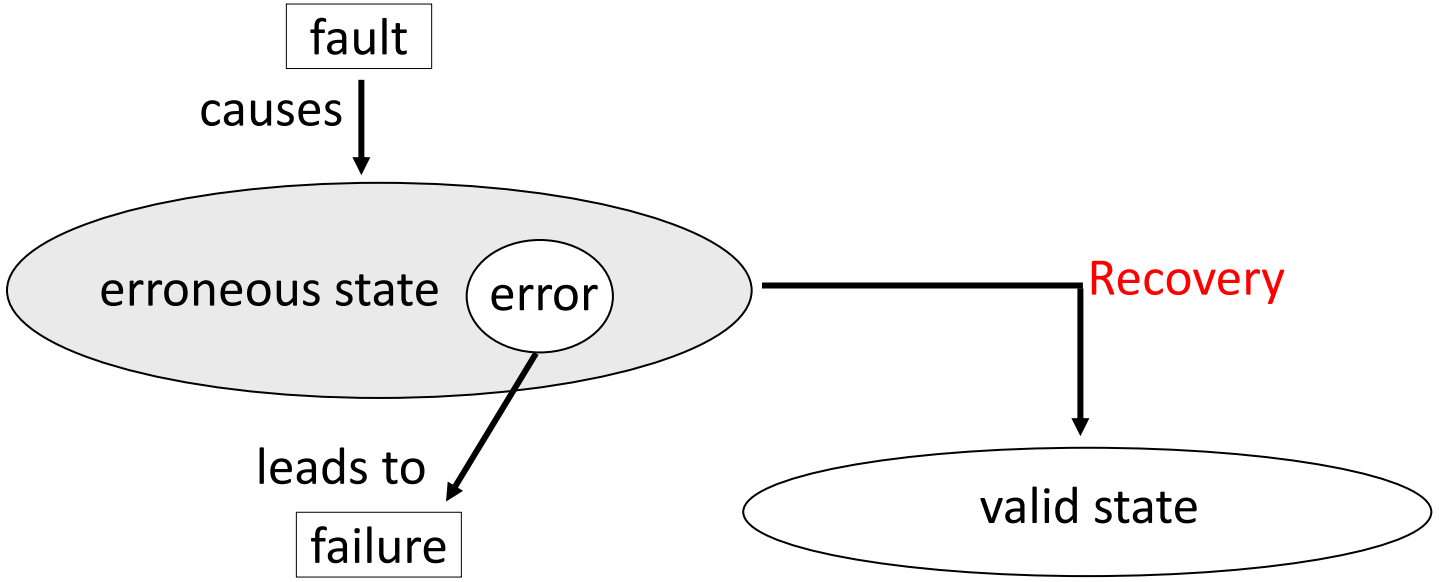
(Fault Tolerance in Distributed Systems)

2110473 Fault Tolerant Computing

Contents

- Fault Tolerance & Recovery
- Rollback Recovery
- System Model:
  - Messages
  - Consistent State (Cuts)
- Rollback Issues:
  - Domino Effect
  - Livelocks
- Recovery Techniques
  - Coordinated checkpointing
  - Pessimistic Logging

Fault Tolerance & Recovery



- An error is a manifestation of a fault that can lead to a failure.
- Failures:
  - Process failure: Deadlocks, protection violation, erroneous user input.
  - System failure: Full/partial amnesia.
  - Secondary storage failure: data inaccessible.
  - Communication failure: network inaccessible.

Recovery

Failure of a site/node in a distributed system causes inconsistencies in the state of the system



Recovery: bringing back the failed node in step with other nodes in the system

- Restart process/processor
- Reclaim resources
- Undo/finish incomplete transactions

Distributed/Concurrency make things harder

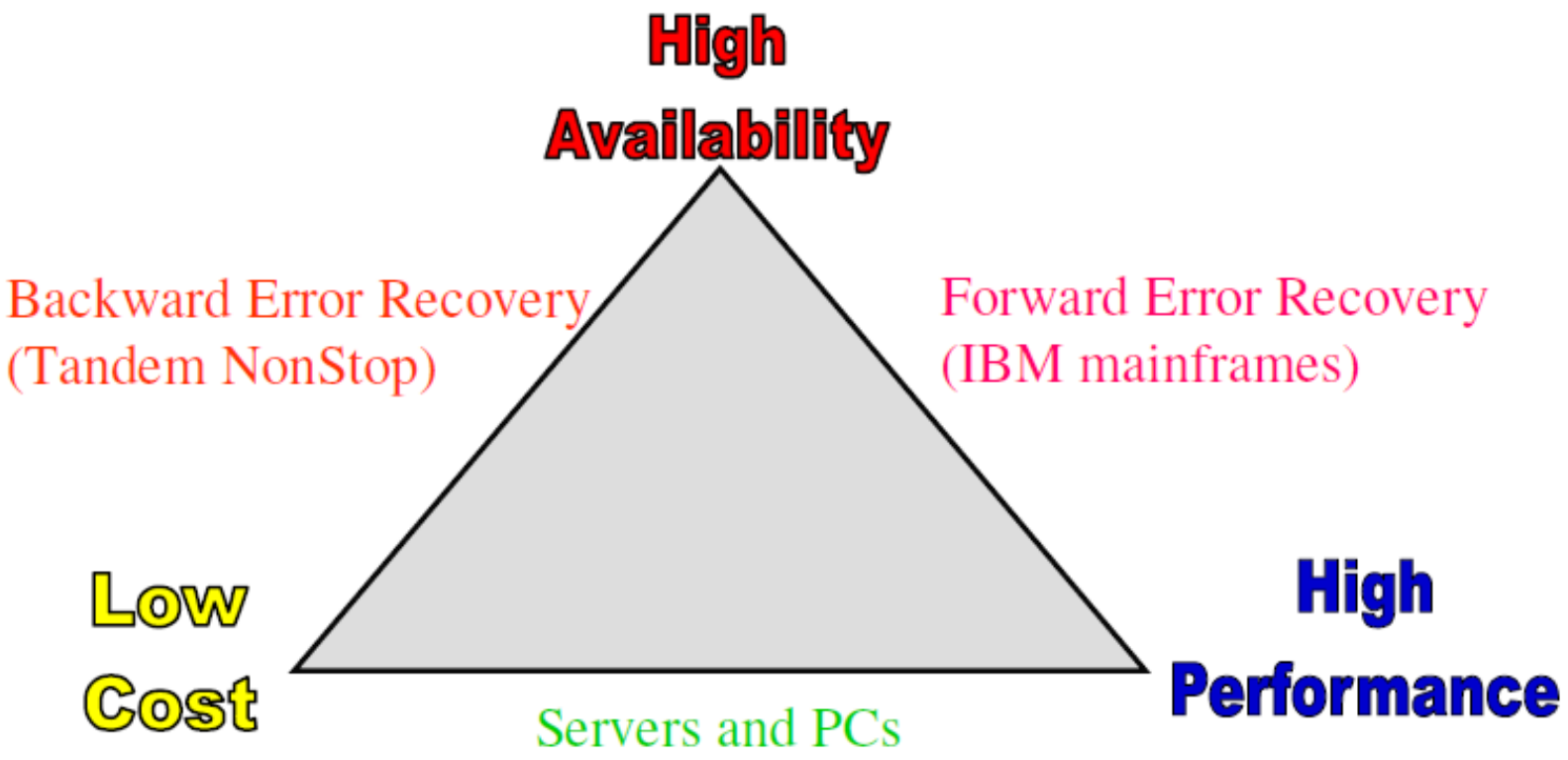
Focus on Message-Passing Model in Distributed Systems

Failure Recovery

After a failure, restoring the system to its "normal" state

- Forward recovery
  - Assess damages that could be caused by faults, remove those damages (errors), and help processes continue
  - TMR is one of Forward recovery
  - Sacrifices cost to achieve availability
- Backward (Rollback) recovery
  - Operation-based (do/undo/redo logs)
  - State-based (Checkpointing/Logging)
  - Sacrifices performance to achieve availability

System Design Space



Forward Recovery

- Repair and go ahead
- Goal: To restore system from erroneous state to error-free state
- If nature of error is completely known
  - Remove error from state
  - Proceed with execution from error-free state
- Rarely possible to do

Forward Recovery (Exception)

- Exceptions
  - System states that should not occur
  - Exceptions can be defined either
    - predefined (e.g. array-index out of bounds, divide by zero)
    - explicitly declared by the programmer
- Raising an exception
  - When such a state is detected in the execution of the program
  - The action of indicating occurrence of such as state
- Exception handler
  - Code to be executed when an exception is raised
  - Declared by the programmer
  - For recovery action
- Supported by several programming languages
  - Ada, ISO Modula-2, Delphi, Java, C++.

Backward/Rollback Recovery

- Repairing is not possible so Go back!
- When error source unknown
  - Restore state to previous error-free state; restart
  - Independent of fault, errors causing fault
  - Problems
    - Performance penalty
    - No guarantee fault will not reoccur
    - Possible unrecoverable component of state
- Recovery point: state used to replace error
- Can not be applied to irreversible operations, e.g. Print out, ATM withdrawal

Basic of Rollback Recovery

- Rollback Recovery Protocols
  - Restore the system back to a Consistent State after a failure
  - Achieve Fault Tolerance by periodically saving the state of a process during the failure-free execution
  - Treat a distributed system application as a collection of processes that communicate over a network
- Checkpoints
  - The Saved States of a process
- Why Rollback Recovery is complicated?
  - Messages induce inter-process dependencies during failure-free operation
- Rollback Propagation
  - The dependencies may force some of the processes that did not fail to roll back
  - This phenomenon is called "Domino Effect"

Rollback Recovery

- State based
  - Checkpointing
    - Uncoordinated checkpointing
    - Coordinated checkpointing
      - Blocking, Non-blocking
    - Communication induced checkpointing
      - Model based, Index based
- Log based
  - Pessimisstic logging
  - Optimistic logging
  - Casual logging

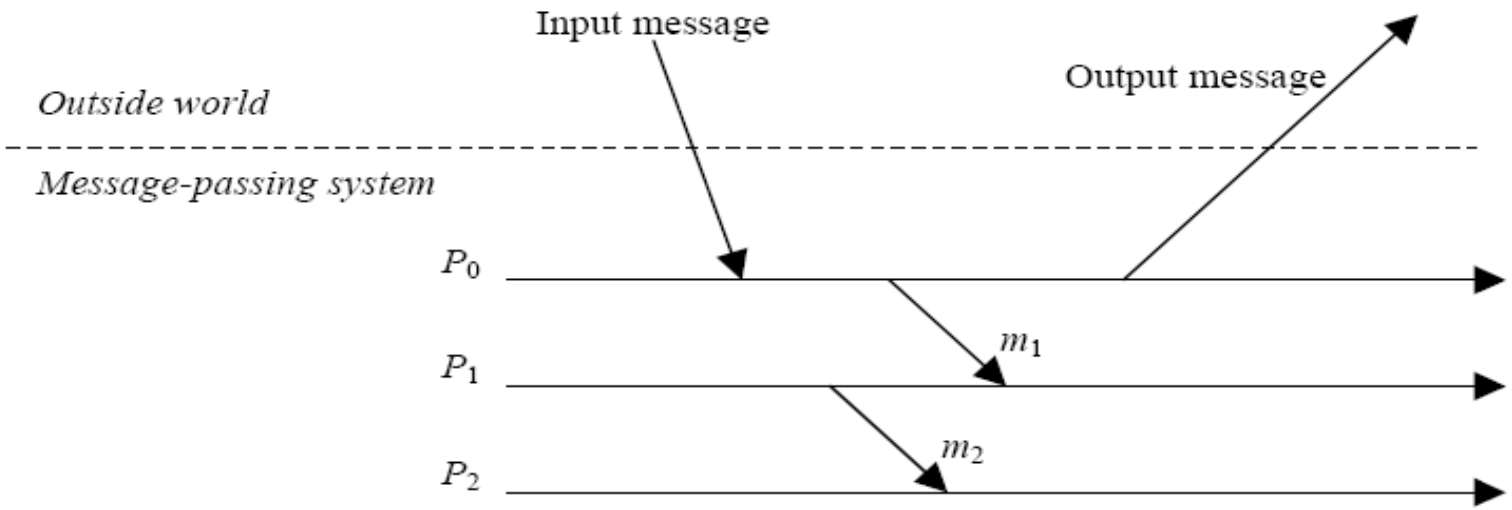
Checkpointing

- Checkpoint recover is the saving and restoration of the system state.
- By saving the current state of the system periodically, it provides the information needed for the restoration of the lost state, when the system fails

Checkpointing issues in Distributed Environment

- Consistency: if P1 checkpoints and then sends a message to P2, we can no longer roll P1 back to before the message was sent unless P2 rolls back to before the message's receipt
- Expensive: We don't want checkpoint more than necessary, but also don't want to have to rollback too far
- Communication with the outside world doesn't get undone

System Model



Basic approaches

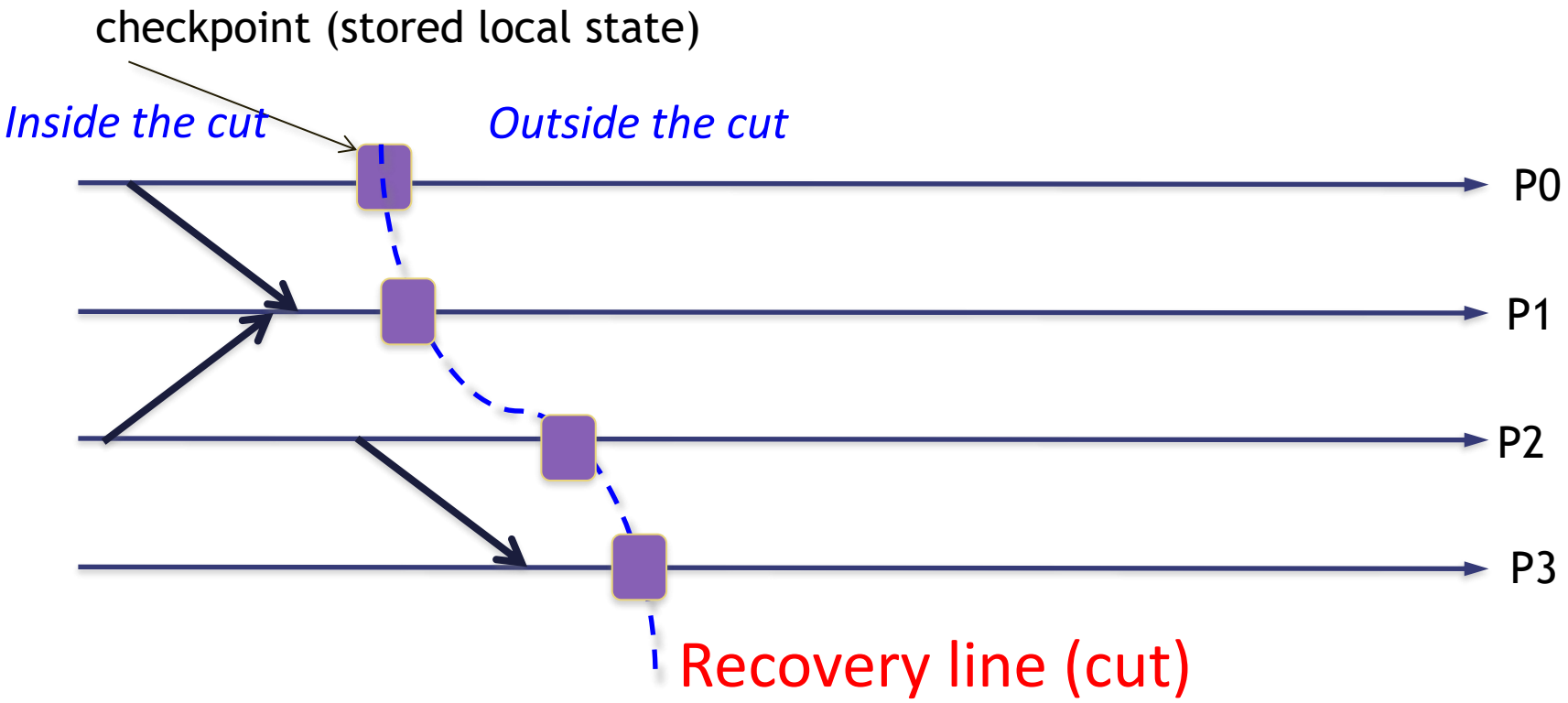
- Checkpointing : copying/restoring the state of a process
- Logging : recording/replaying messages

Stable storage – a prerequisite for recovery

- In a system that tolerates only *a single failure*, stable storage may consist of **the volatile memory of another process**
- In a system that wishes to tolerate an arbitrary number of *transient failures, stable* storage may consist of **a local disk in each host**.
- In a system that tolerates *non-transient failures*, stable storage must consist of a **persistent medium outside the host on which a process is running**. A **replicated file system** is a possible implementation in such systems

Rollback Recovery Principles

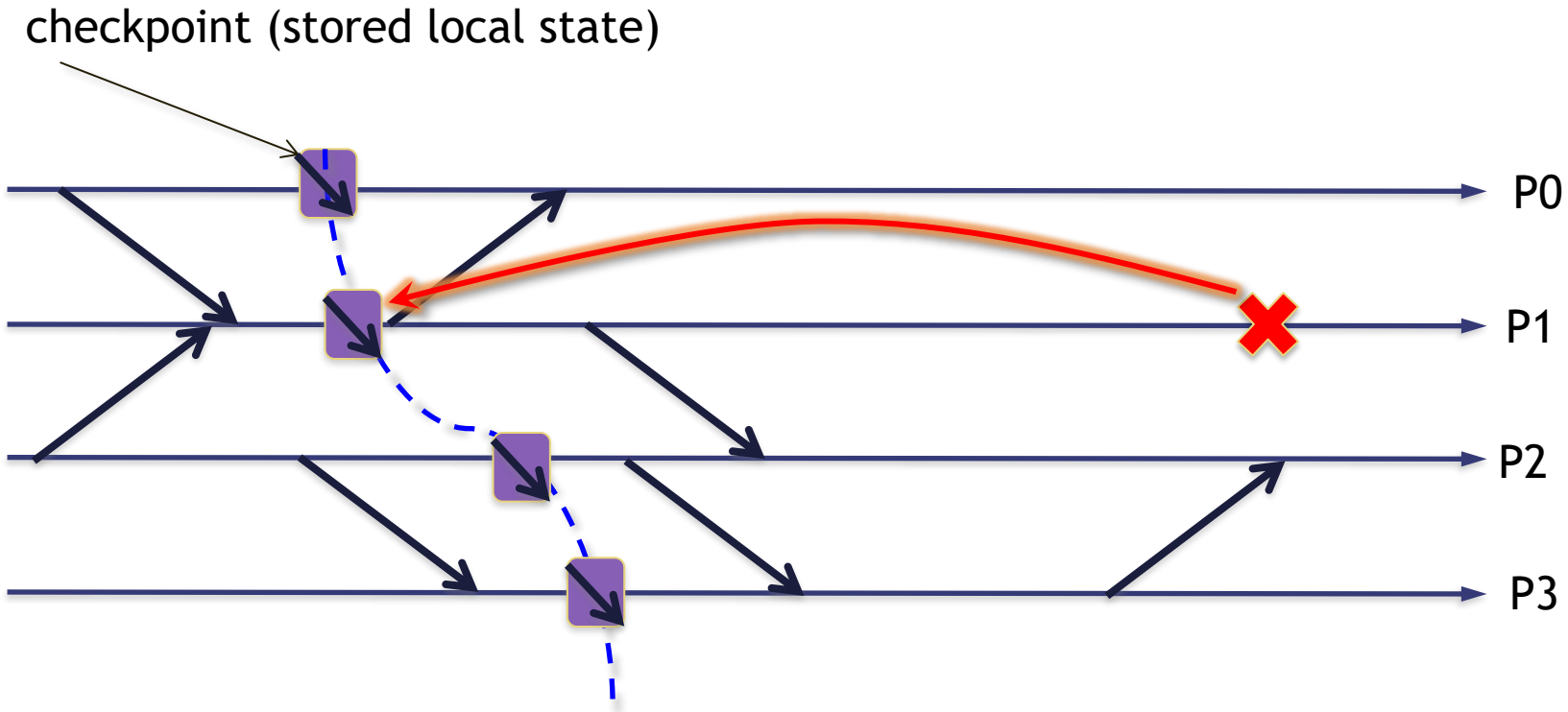
1. Checkpointing



Restart all, or almost all, processes from a consistent cut and let a new execution run

Rollback Recovery Principles

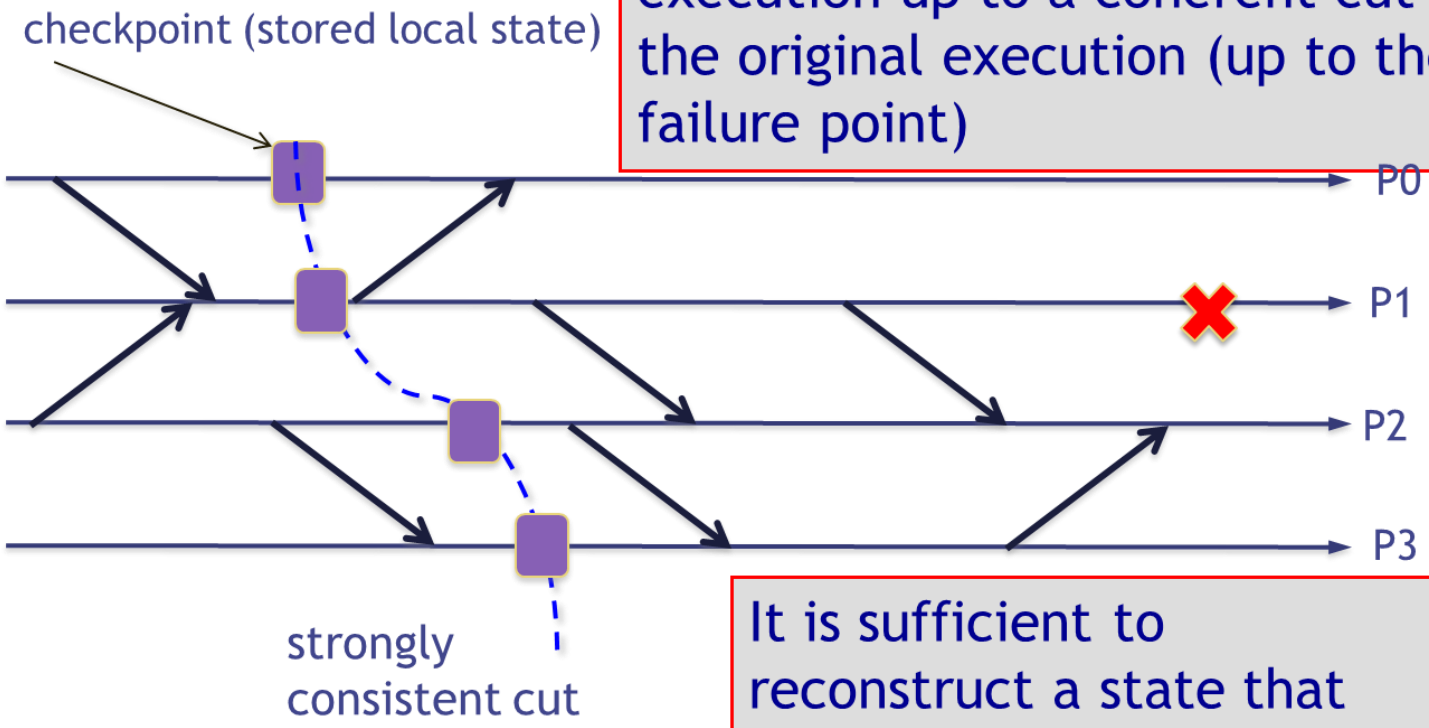
2. Message Logging



Only one (or a few) process recover and use message information to replay the previous execution until reaching the failure point

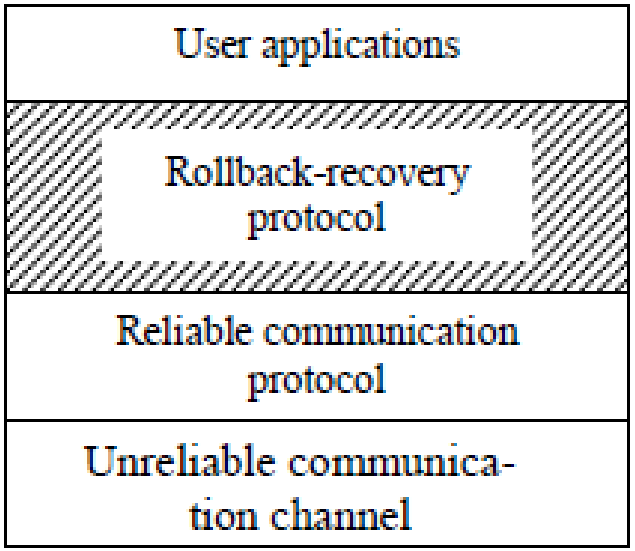
Rollback Recovery Principles

A recoverable state contains enough information to replay an execution up to a coherent cut of the original execution (up to the failure point)

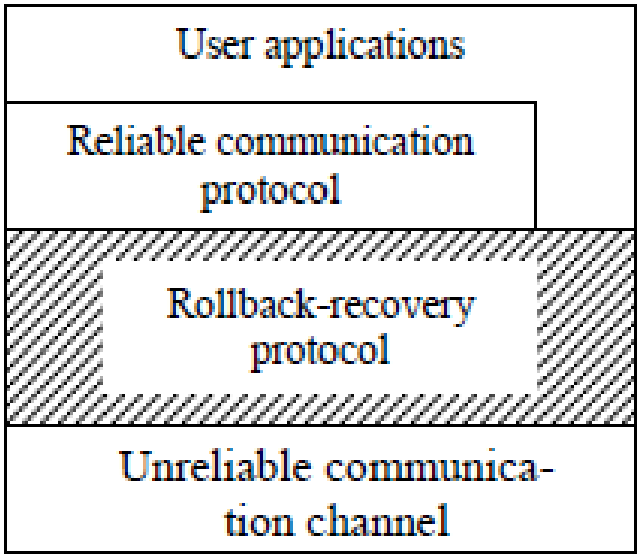


It is sufficient to reconstruct a state that *could* have occurred in a failure-free execution

Implementation of Rollback Recovery



On top of a reliable communication protocol



Directly on top of unreliable communication channels

Garbage Collection

- Checkpoints and event logs consume storage resources
- Stored Information may become useless for Recovery
- Garbage Collection:
  - The deletion of such useless recovery information
  - Common approach is to identify the most recent consistent set of checkpoints (Recovery line)
  - And discard all information relating to events that occurred before that line

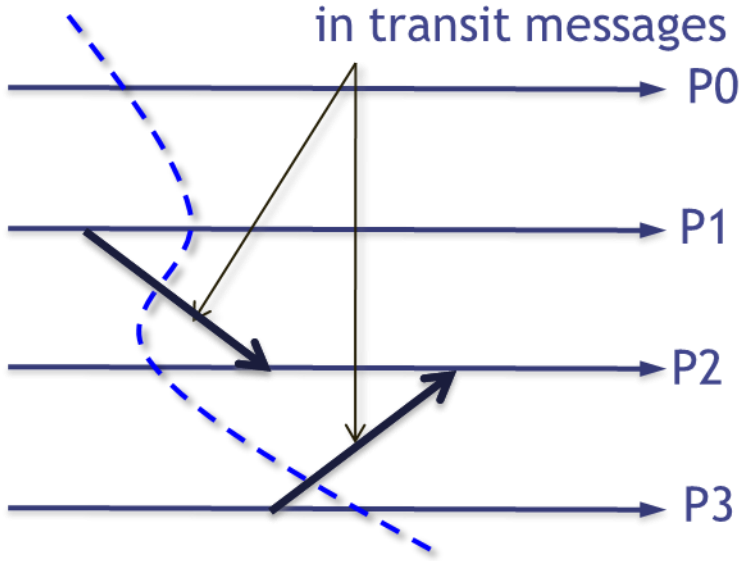
Messages

- In-transit message
  - messages that have been sent but not yet received
- Lost messages
  - messages whose ‘send’ is done but ‘receive’ is undone due to rollback
- Delayed messages
  - messages whose ‘receive’ is not recorded because the receiving process was either down or the message arrived after rollback
- Orphan messages
  - messages with ‘receive’ recorded but message ‘send’ not recorded
  - do not arise if processes roll back to a consistent global state
- Duplicate messages
  - arise due to message logging and replaying during process recovery

In-transit message

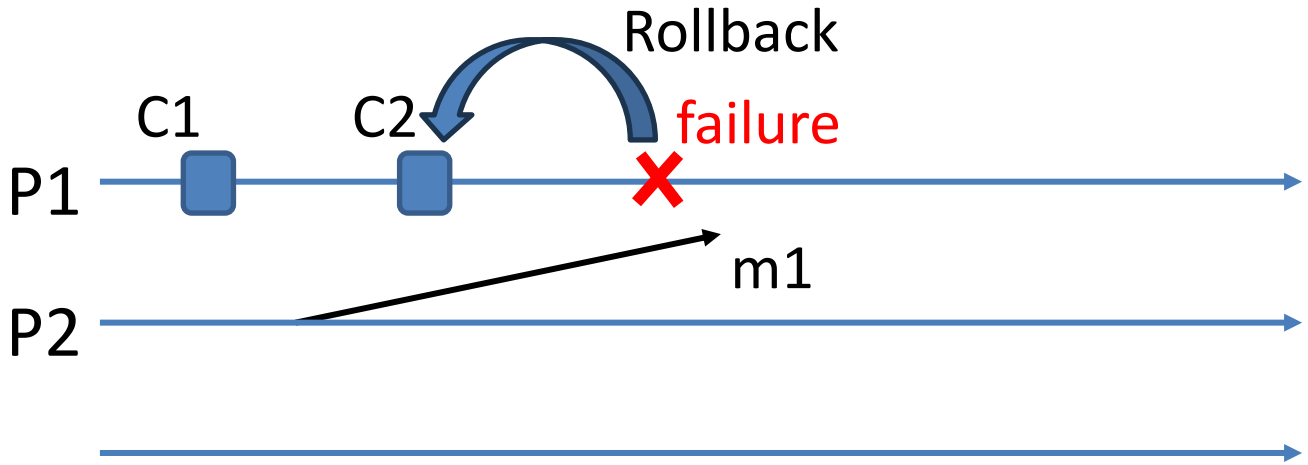
messages that have been sent but not yet received

- If message delivery is not guaranteed, they are not a problem!
- But if the communication protocol is reliable, they must be taken into account



Lost message

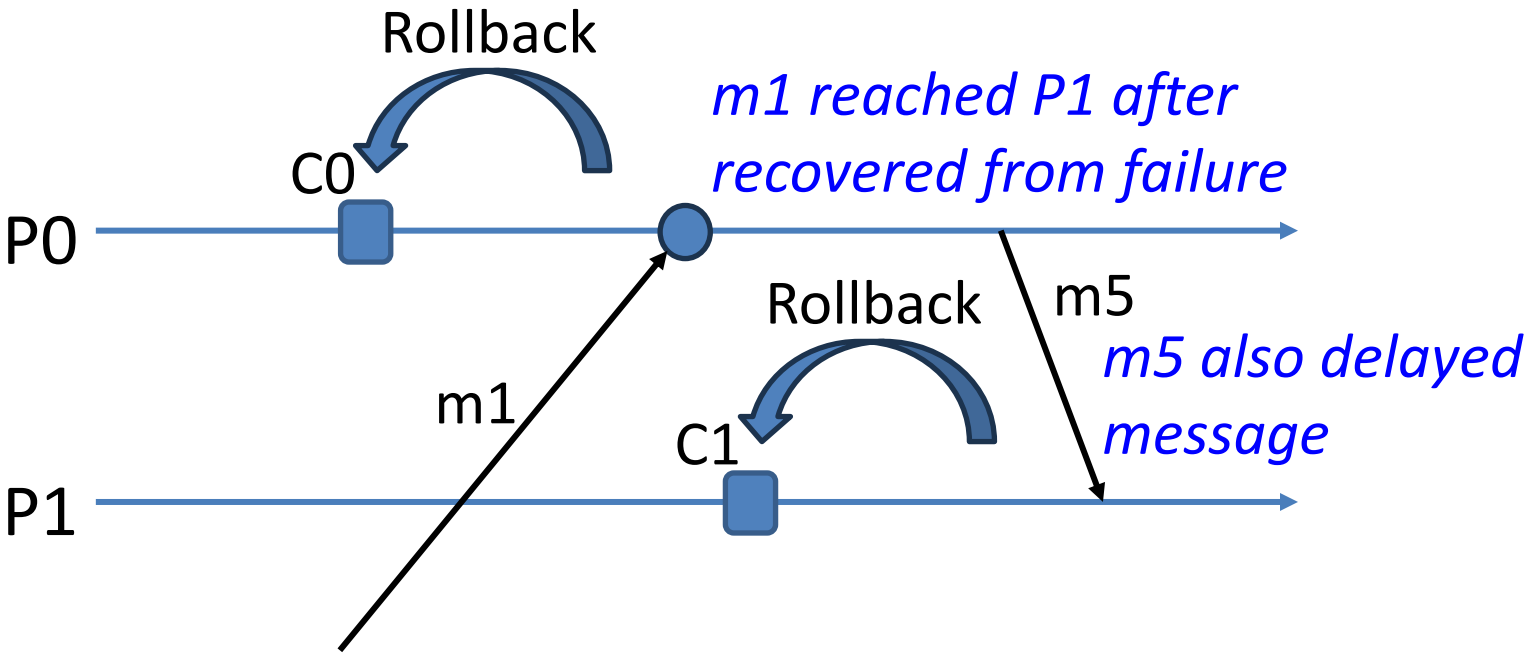
- Due to the failure in process, the message cound not reach the process and so it is lost on the way
- Different from In-transit message



- Due to the failure in **P1**, m1 could not reach **P1** and lost on the way
- **P1** rollback to **C2** to recover its old status to continue the task

Delayed message

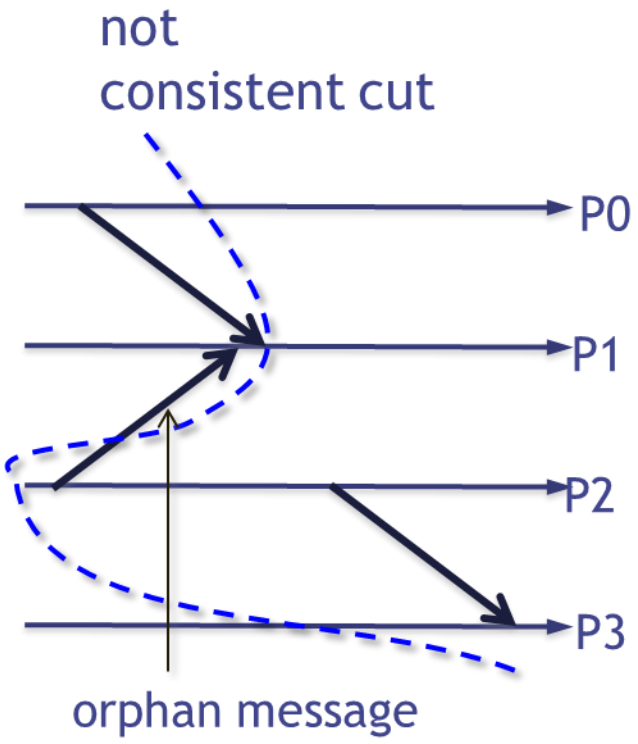
- The lost messages may reach the destination process after recovered, ie. messages arrived after rollback



Orphan messages

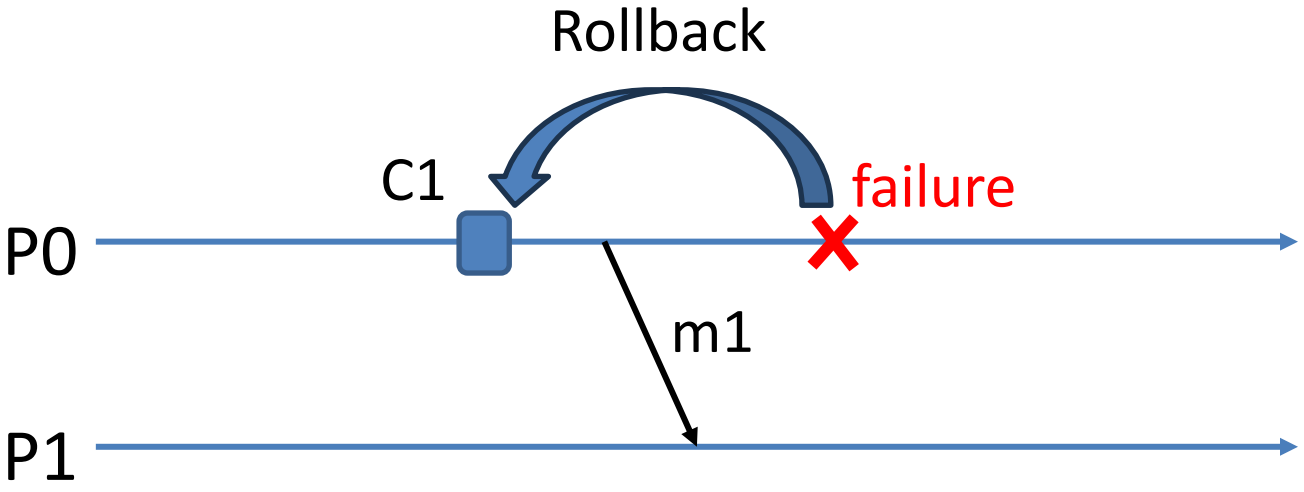
messages with 'receive' recorded but message 'send' not recorded

- If P2 fails and restarts from the cut, the message will be re-emitted and received twice by P1
  - Either avoid using inconsistent cuts (in general for checkpointing)
  - Or avoid re-emitting the message (in general for message logging) **and replay the same execution**



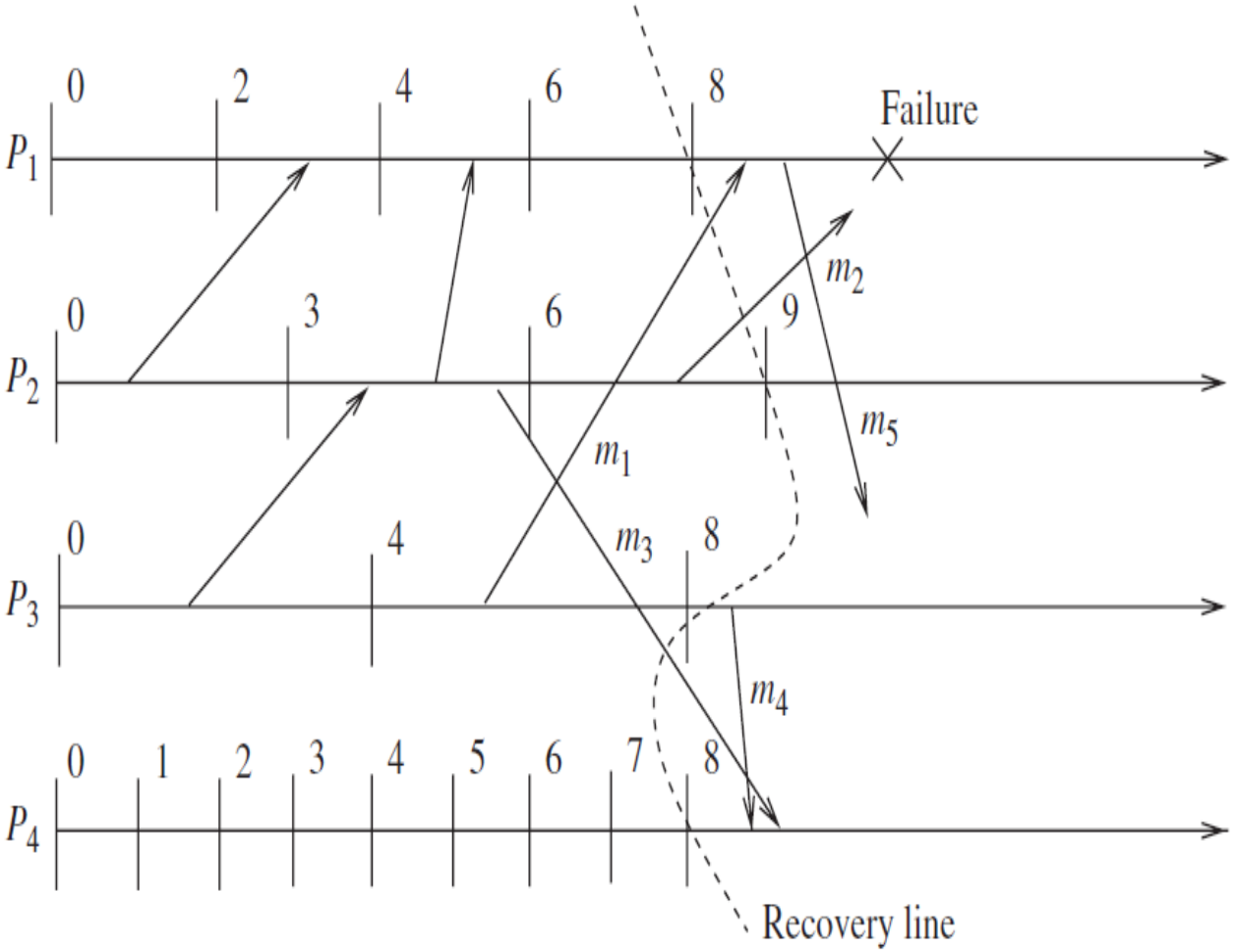
Duplicate message

- The message which already received has been re-sent after recovery



Messages – example

- In-transit
  - $m1, m2$
- Lost
  - $m1$
- Delayed
  - $m1, m5$
- Orphan
  - none
- Duplicated
  - $m4, m5$

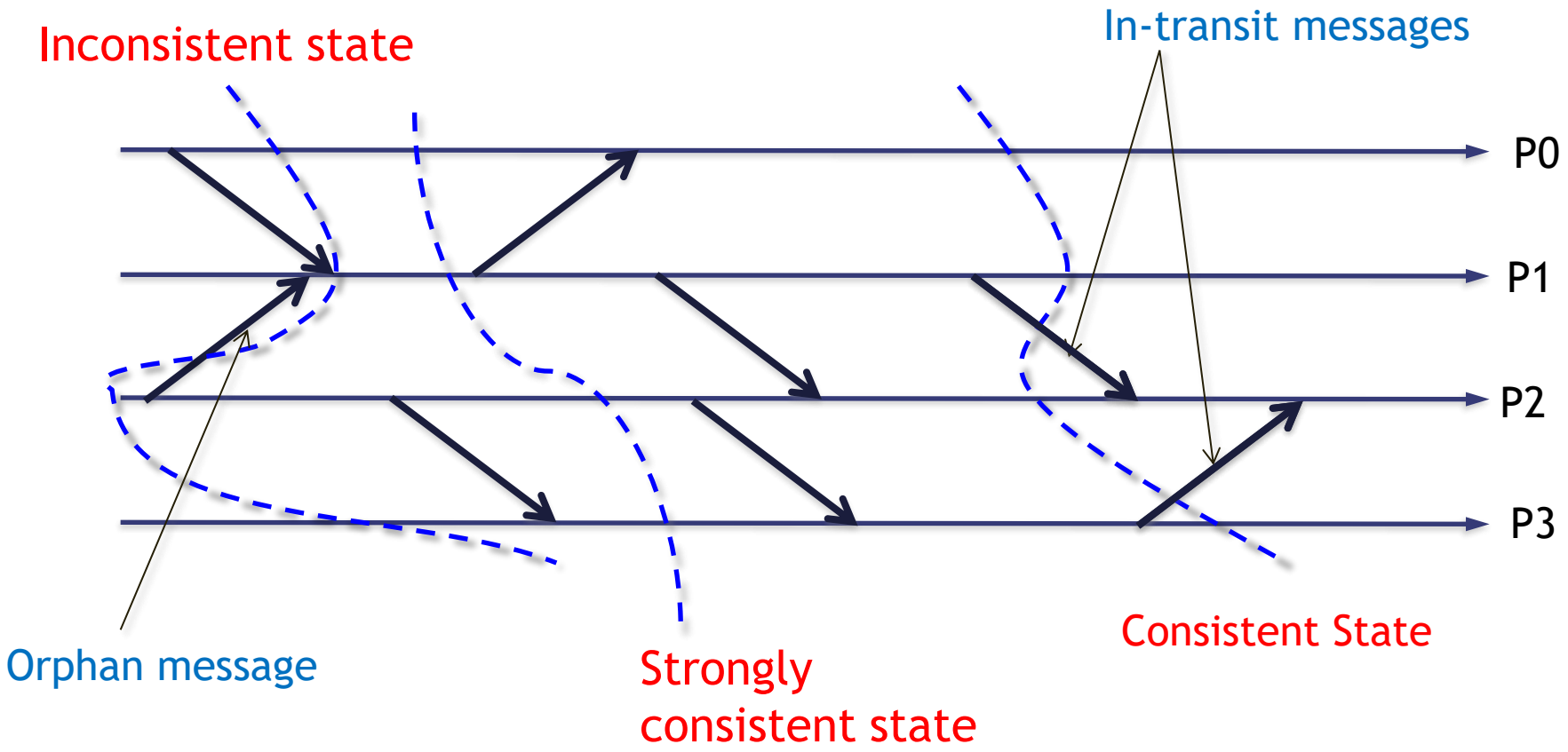


Consistent State

- A global state of a distributed system
  - a collection of the individual states of all participating processes and the states of the communication channels
- Consistent global state
  - a global state that may occur during a failure-free execution of distribution of distributed computation
  - if a process's state reflects a message receipt, then the state of the corresponding sender must reflect the sending of the message
- A global checkpoint
  - a set of local checkpoints, one from each process
- A consistent global checkpoint
  - a global checkpoint such that no message is sent by a process after taking its local point that is received by another process before taking its checkpoint

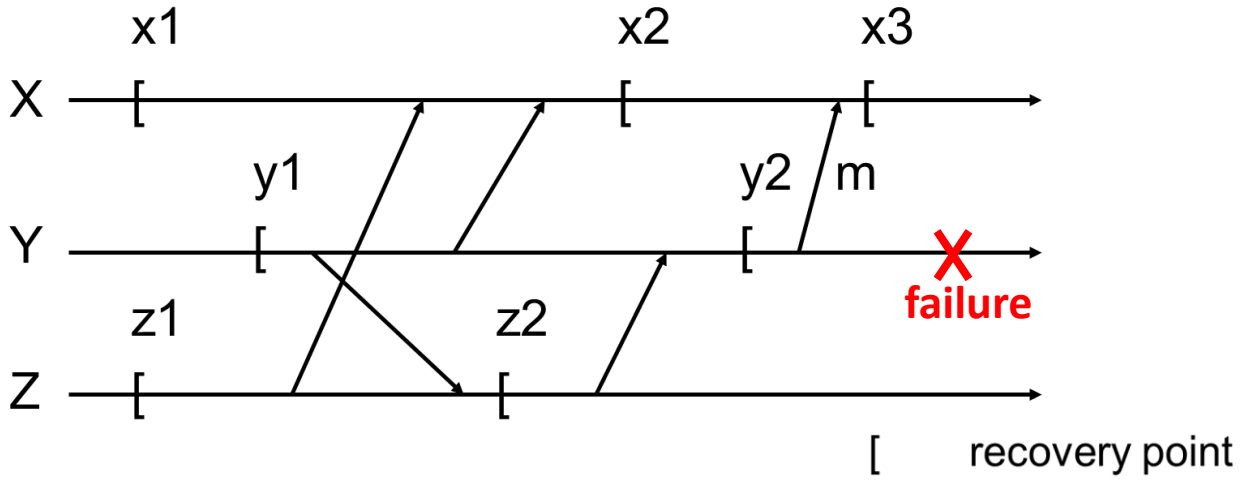
"A fundamental goal of any rollback-recovery protocol is to bring the system into a consistent state when inconsistencies occur because of a failure"

Consistent/Inconsistent State



Domino Effect

Cascaded rollback which causes the system to roll back to too far in the computation (even to the beginning), in spite of all the checkpoints



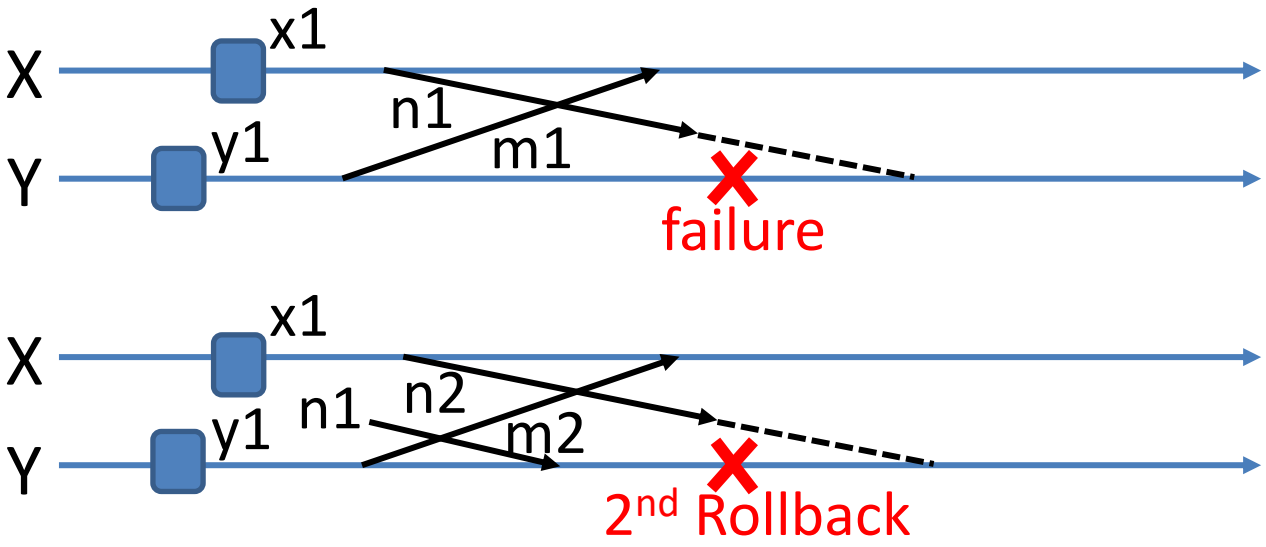
Suppose Y rolls back to y2

- m is orphan message
- Process X must rollback to x2

Suppose Z rolls back to z2

- Y rolls back to y1
- Force X, Z to roll back to x1, z1

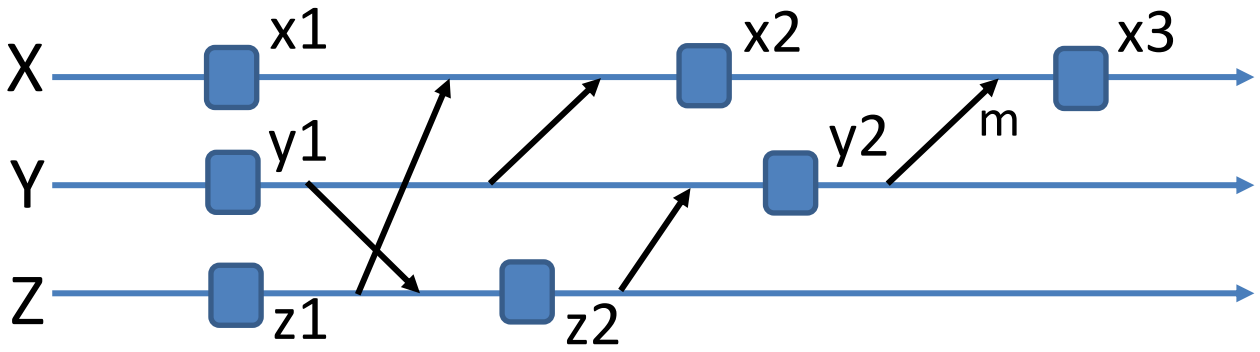
Livelocks



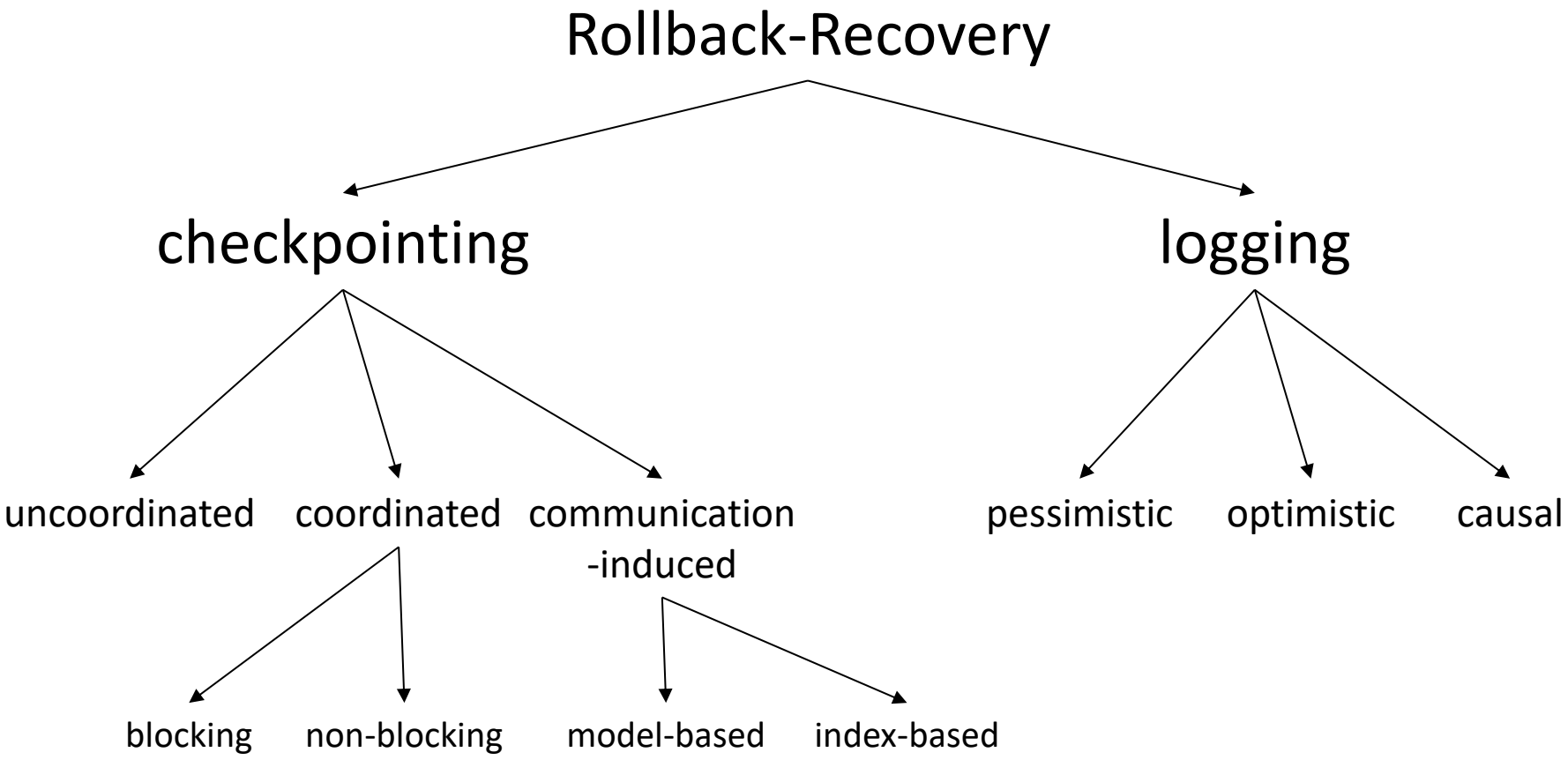
- Y crashes before receiving n1. Y rolls back to Y1 -> X to x1.
- Y recovers, receives n1 and sends m2.
- X recovers, sends n2 but has no record of sending n1
- Hence, Y is forced to rollback second time. X also rolls back as it has received m2 but Y has no record of m2.
- Above sequence can repeat indefinitely, causing a livelock.

Consistent Checkpoints

- Overcoming domino effect and livelocks: checkpoints should not have messages in transit.
- Consistent checkpoints: no message exchange between any pair of processes in the set as well as outside the set during the interval spanned by checkpoints.
- {x1,y1,z1} is a strongly consistent checkpoint.



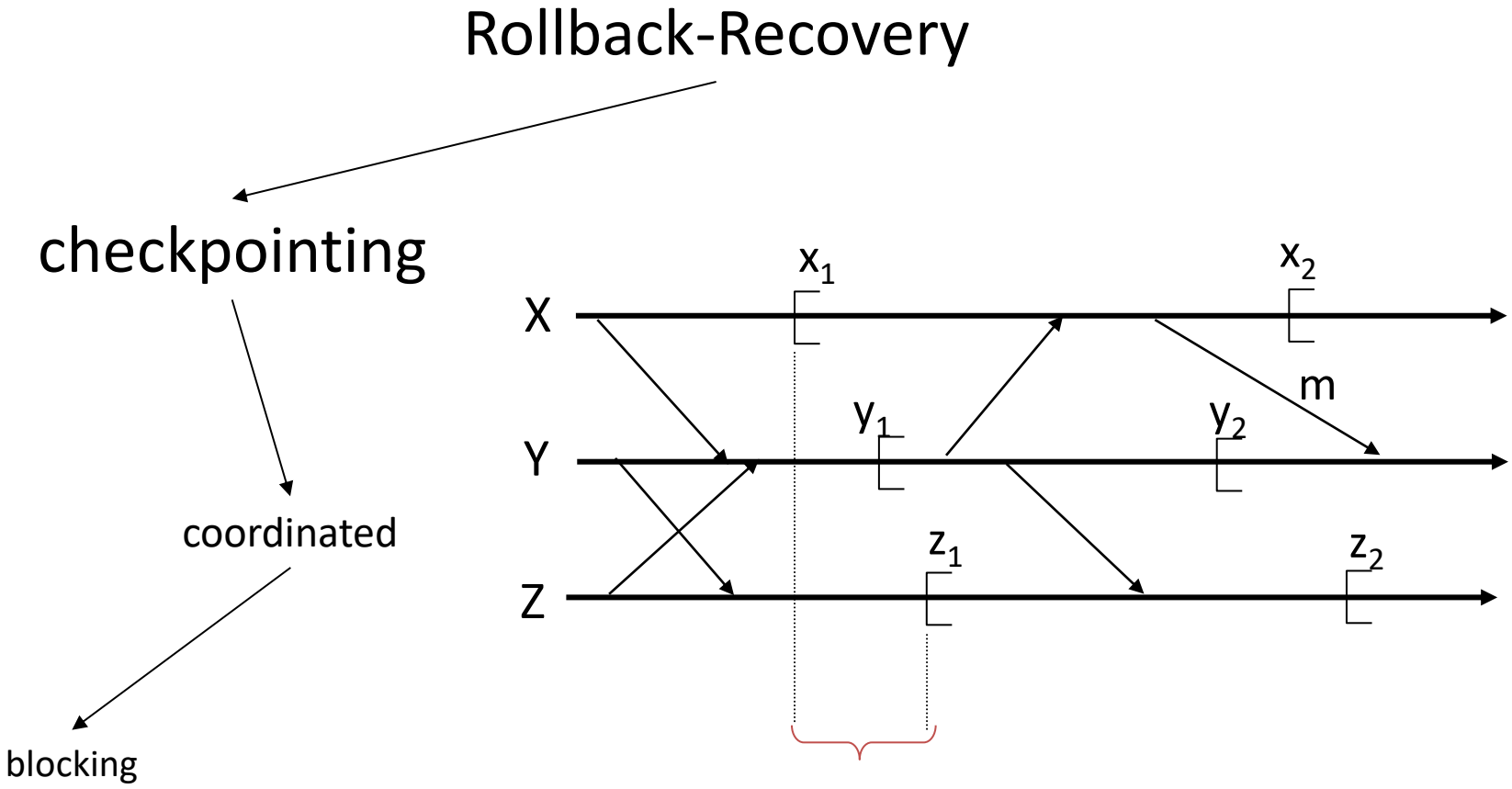
Recovery Techniques



Techniques that avoid Domino effect

- Coordinated checkpointing rollback recovery
  - processes coordinate their checkpoints to form a system-wide consistent state
- Communication-induced checkpointing rollback recovery
  - forces each process to take checkpoints based on information piggybacked on the application
- Log-based rollback recovery
  - combines checkpointing with logging of non-deterministic events
  - relies on piecewise deterministic (PWD) assumption

Coordinated/Blocking Protocols



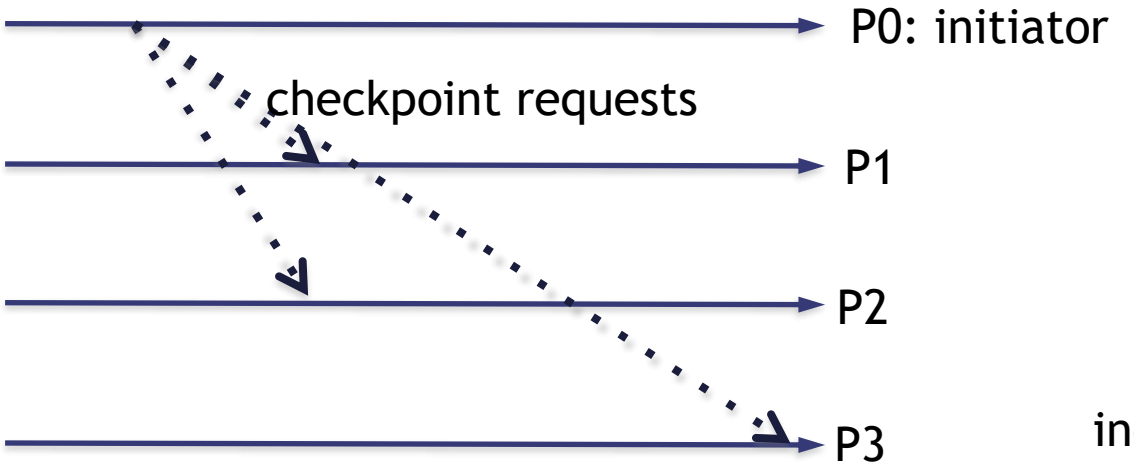
- no messages can be in transit during checkpointing
- {x1, y1, z1} forms “recovery line”

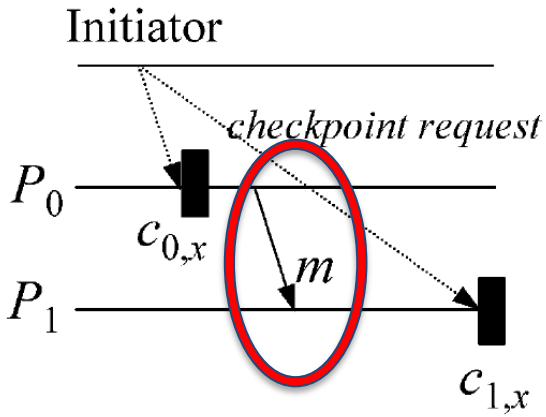
Blocking & Non-blocking Coordinated Checkpointing

- Blocking Checkpointing
  - After a process takes a local checkpoint, to prevent orphan messages, it remains blocked until the entire checkpointing activity is complete
  - Disadvantages
    - the computation is blocked during the checkpointing
- Non-blocking Checkpointing
  - The processes need not stop their execution while taking checkpoints
  - A fundamental problem in coordinated checkpointing is to prevent a process from receiving application messages that could make the checkpoint inconsistent.

Coordinated checkpointing

- There is an initiator process for the checkpointing
  - Only one (or 2) checkpoint per process (always consistent)
  - large latency: processed blocked until checkpoint is finished



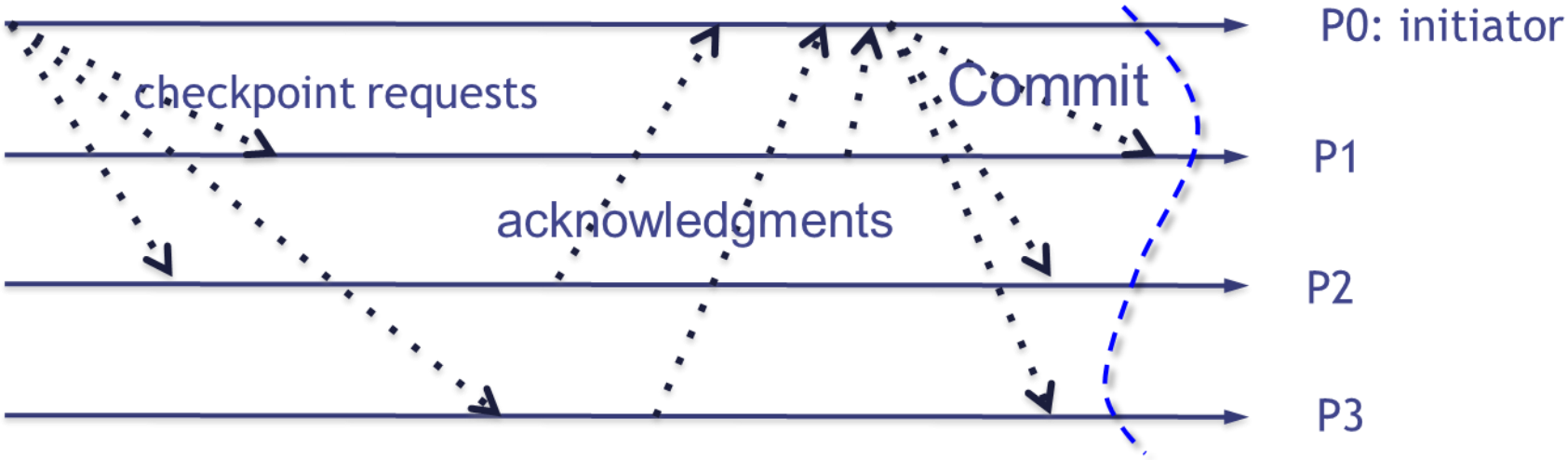


inconsistency if communications are not blocked until the end of the checkpointing phase

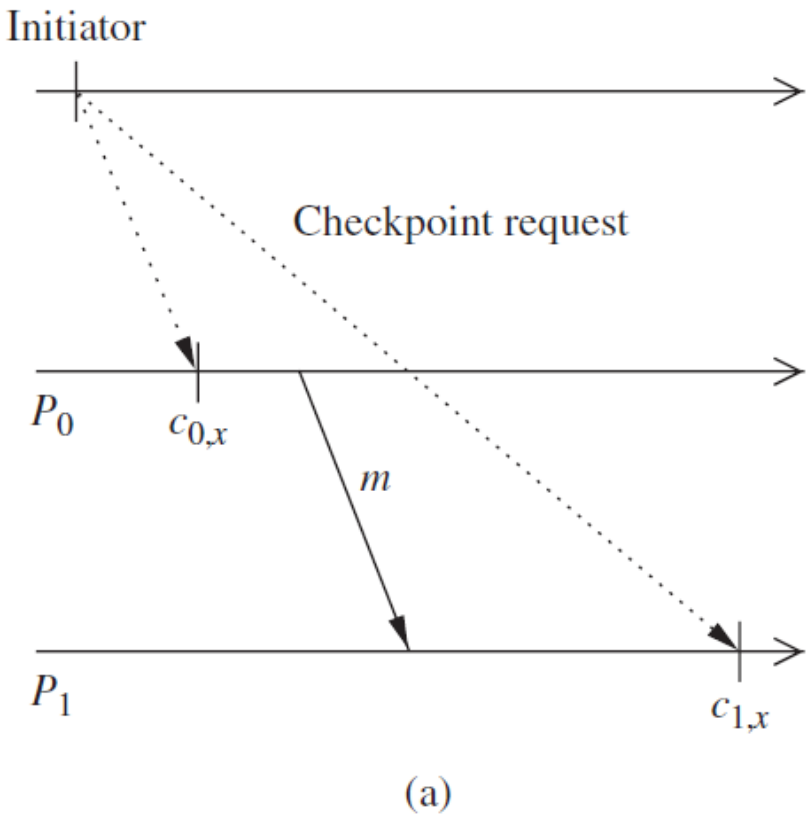
Coordinated checkpointing (2)

- Algorithm:
  - block communications while the protocol executes
  - An initiator takes a checkpoint and broadcasts a request message to all processes
  - When a process receives this message, it
    - stops its execution,
    - flushes all the communication channels,
    - takes a *tentative checkpoint, and*
    - sends an acknowledgment message back
  - the coordinator receives acknowledgments from all processes, and broadcasts a commit message
  - After receiving the commit each process removes the old checkpoint, the new one becomes permanent

Coordinated Checkpointing (3)
Overall execution graph



Checkpoint inconsistency

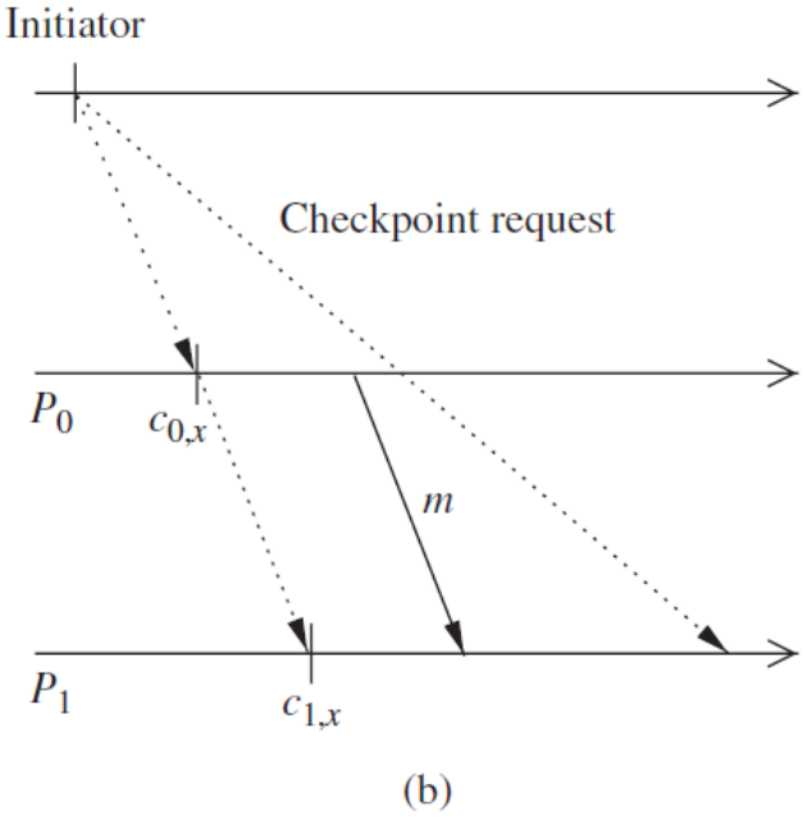


(a)

- message m is sent by P_0 after receiving a checkpoint request from the checkpoint coordinator
- Assume m reaches P_1 before the checkpoint request
- This situation results in an inconsistent checkpoint since checkpoint $C_{1,x}$ shows the receipt of message m from P_0, while checkpoint $C_{0,x}$ does not show m being sent from P_0

Checkpoint inconsistency: a solution with FIFO channels

- If channels are FIFO, this problem can be avoided by preceding the first post-checkpoint message on each channel by a checkpoint request, forcing each process to take a checkpoint before receiving the first post-checkpoint message



(b)

Uncoordinated checkpointing

- Each process has autonomy in deciding when to take checkpoints
- Advantages
  - The lower runtime overhead during normal execution
- Disadvantages
  - Domino effect during a recovery
  - Recovery from a failure is slow because processes need to iterate to find a consistent set of checkpoints
  - Each process maintains multiple checkpoints and periodically invoke a garbage collection algorithm
  - Not suitable for application with frequent output commits
- The processes record the dependencies among their checkpoints caused by message exchange during failure-free operation

Communication-induced Checkpointing

- Two types of checkpoints
  - autonomous and forced checkpoints
- Communication-induced checkpointing piggybacks protocol- related information on each application message
- The receiver of each application message uses the piggybacked information to determine if it has to take a forced checkpoint to advance the global recovery line
- The forced checkpoint must be taken before the application may process the contents of the message
- In contrast with coordinated checkpointing, no special coordination messages are exchanged
- Two types of communication-induced checkpointing
  - model-based checkpointing and index-based checkpointing.

Log-based Rollback Recovery

- A log-based rollback recovery makes use of deterministic and nondeterministic events in a computation.
- Deterministic and Non-deterministic events
  - Non-deterministic events can be the receipt of a message from another process or an event internal to the process
  - a message send event is *not* a non-deterministic event.
  - the execution of process $P0$ is a sequence of four deterministic intervals
  - Log-based rollback recovery assumes that all non-deterministic events can be identified and their corresponding determinants can be logged into the stable storage
  - During failure-free operation, each process logs the determinants of all non-deterministic events that it observes onto the stable storage

Pessimistic Logging

- ***piecewise determinism*** = all non-deterministic events can be identified and their determinants can be stored on stable storage.
- An execution is a sequence of deterministic events (replayed) and non-deterministic events (logged and simulated from log)
- determinants of non-deterministic events are stored during failure-free execution
- + checkpoints to avoid recovering from the start
- Additional hypothesis: It is possible to prevent a message from being sent or received

Pessimistic Logging

- A ***process is orphan*** if it depends on the execution of a non-logged non-deterministic event
- ***Determinant:*** the information need to "replay" the occurrence of a non-deterministic event (e.g., message reception)
- ***always-no-orphans condition:***
  - Stable(e) set of processes with e logged on stable storage
  - Depend(e) set of processes affected by event e
  - Log(e) = set of processes locally storing the event e

$$\forall e : \neg Stable(e) \Rightarrow Depend(e) \subseteq Log(e)$$

else the process is said orphan

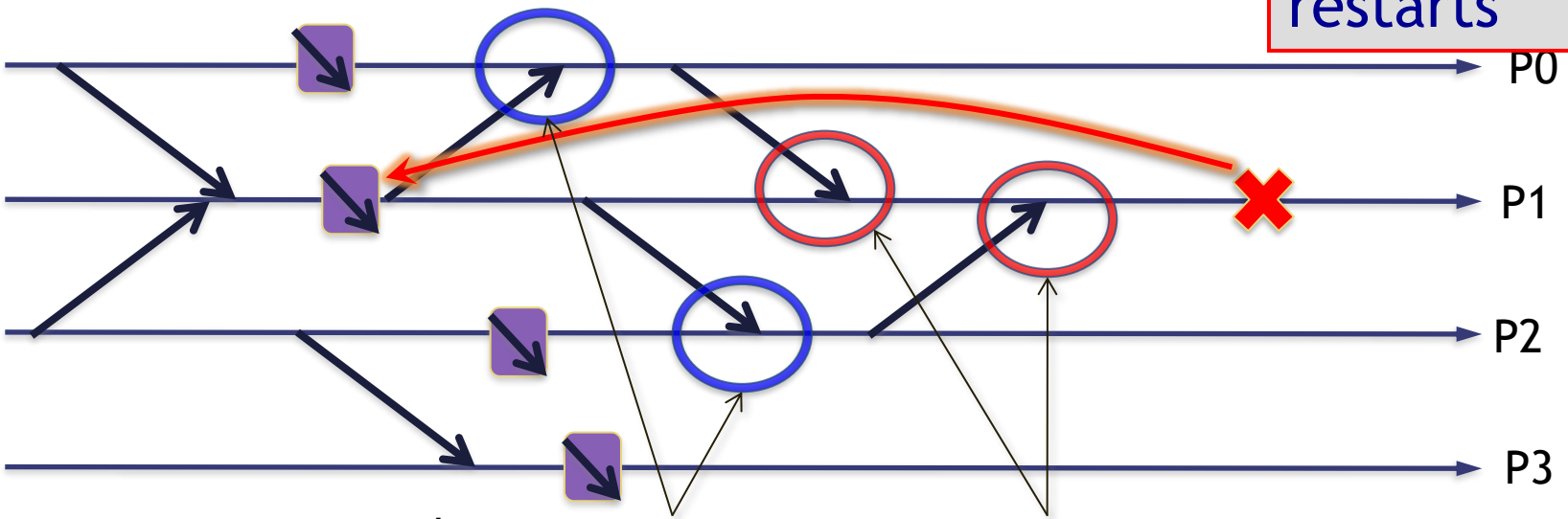
Pessimistic message logging

- orphan processes are never created but requires a lot of synchronizations with the stable storage
- Logs the determinant of ND events before executing them

synchronous logging (stronger condition)

$$\forall e : \neg Stable(e) \Rightarrow |Depend(e)| = 0$$

Only 1 process restarts



Other Issues

- Output commit
  - the state from which messages are sent to the “outside world” can be recovered
  - affects latency of message delivery to “outside world” and overhead of checkpoint/logging
- Stable storage
  - survives process failures
  - contains checkpoint/logging information
- Garbage collection
  - removal of checkpoints/logs no longer needed

HW04: Checkpointing

- Find the consistent cuts in the figure below (passing by )
- The recovery line that you found is consistent, strongly consistent or inconsistent?

